

SOUTHEAST UNIVERSITY

Department of Computer Science and Engineering

PROJECT LAB REPORT

WorkLedger

A Web-Based Labour Attendance and Payroll Management System

Built with Next.js, Tailwind CSS, Prisma ORM and PostgreSQL

SUBMITTED BY

Name	MD Shishier Ahamed
	Shuvo Tamim
	Fatema Jaman
Student ID	2022100000035
	2023000000181
	2023100000485
Department	Computer Science and Engineering
University	Southeast University
Course Title & Code	[Software Development and Project Management Lab & 474.3]
Semester	[12]

SUBMITTED TO

Course Lecturer	[Dr. Rakib Ahmed Saleh]
Designation	[Lecturer,Southeast University]
Department	Computer Science and Engineering, Southeast University

Date of Submission: [10/2/2026]

Table of Contents & Document Structure Overview

Document Structure Overview

This report is organised into front matter, ten technical chapters and a closing section. The overview below maps each part of the report to its purpose so that academic evaluators, prospective buyers and investors can navigate directly to the material most relevant to them.

Table 1: Report structure and intended audience

Part	Content	Primary Audience
Front Matter	Cover page, table of contents, acknowledgement and dedication	Academic evaluators
Executive Summary	Value proposition, target market and headline outcomes of WorkLedger	Investors, buyers
Chapter 1	Introduction, background, problem statement, objectives and scope	All readers
Chapter 2	Literature review, comparative analysis and feasibility study	Evaluators, investors
Chapter 3	Functional and non-functional requirements	Evaluators, developers
Chapters 4–5	System architecture, data flow and PostgreSQL / Prisma schema design	Evaluators, technical buyers
Chapter 6	RBAC, attendance module and payroll module implementation	Evaluators, developers
Chapter 7	Technology stack rationale and performance optimisation	Technical buyers
Chapter 8	Testing, quality assurance and troubleshooting	Evaluators, developers
Chapter 9	Business impact, ROI and buyer value	Buyers, investors
Chapter 10	Future roadmap and scalability	Investors, evaluators
Conclusion	Summary, lessons learned	All readers

Acknowledgement & Dedication

Acknowledgement

First and foremost, I express my sincere gratitude to Almighty Allah for granting me the health, patience and perseverance required to complete this project successfully.

I am deeply grateful to my course lecturer, [Dr. Rakib Ahmed Saleh], Department of Computer Science and Engineering, Southeast University, for continuous guidance, constructive criticism and encouragement throughout the development of WorkLedger. Their insistence on engineering discipline – clear requirements, normalised data models and systematic testing – shaped both the software and this report.

I also thank the Head of the Department and all faculty members of the Department of Computer Science and Engineering for providing a rigorous academic foundation and access to laboratory resources. My appreciation extends to my classmates and friends who participated in usability sessions and offered candid feedback on the owner and worker interfaces, and to the open-source communities behind Next.js, React, Tailwind CSS, Prisma and PostgreSQL, whose documentation and tooling made it possible for a single student developer to build a production-style system.

Finally, I thank my family for their unwavering support, patience and belief in me throughout my academic journey.

MD Shishier Ahamed

ID: 2022100000035

Dedication

*To my parents, whose sacrifices opened every door I have walked through;
to my teachers, who taught me to build things properly;
and to the millions of hardworking labourers whose daily effort
deserves to be recorded accurately and paid fairly.*

Executive Summary

WorkLedger is a secure, web-based labour attendance and payroll management platform designed for small and medium-sized businesses that employ daily-wage and monthly-wage workers – construction contractors, factories, workshops, farms, retail outlets, security agencies and service businesses. It replaces the traditional paper ledger (the *khata*) and ad-hoc spreadsheets with a single, auditable digital system that records attendance every day, calculates salary automatically and gives every worker transparent access to their own record.

Core Value Proposition

The central promise of WorkLedger is simple: **every working day is recorded once, verified once and paid correctly once**. The platform achieves this through four tightly integrated capabilities:

- **Verified daily attendance** – workers log attendance from a phone-friendly interface; the server applies its own timestamp, enforces one record per worker per day and optionally stores a selfie as visual evidence.
- **Automated payroll** – gross pay, overtime, bonuses, advances and deductions are computed by a deterministic, unit-tested formula, eliminating manual arithmetic errors.
- **Role-based dashboards** – owners get an operational and financial command centre; workers get a focused personal view. Authorisation is enforced on the server, not merely hidden in the interface.
- **Transparent audit trail** – every attendance record, correction and payroll run is persisted in PostgreSQL with timestamps and actor identity, reducing disputes between employer and employee.

Target Market

Table 3: Target market segments

Segment	Typical Profile	Key Need Addressed
Construction & contracting	20–200 daily-wage workers across changing sites	Accurate daily headcount and weekly/monthly settlement
Small manufacturing & workshops	10–100 workers, mixed daily and monthly wages	Overtime tracking and advance recovery
Agriculture & plantations	Seasonal labour with variable attendance	Low-cost, mobile-first logging
Retail, security & services	Shift-based staff, multiple branches	Owner visibility without physical presence

Headline Outcomes

The delivered system is built on Next.js (App Router with Turbopack), Tailwind CSS and Prisma ORM over PostgreSQL. It provides a complete vertical slice – authentication, RBAC, attendance, payroll, reporting – and a clear migration path to mobile applications, cloud-scale

infrastructure and automated SMS/push notifications. Under the illustrative assumptions in Chapter 9, a 40-worker business can recover its setup investment in under two months, while the architecture presented in Chapters 4 and 10 supports growth to multi-tenant SaaS operation.

Chapter 1 – Introduction & Background

1.1 Background

Blue-collar and daily-wage workers form the backbone of the economy in South Asia and many other developing regions. A very large share of this workforce is employed informally or by small enterprises that lack dedicated human-resource departments [14]. In such environments, attendance and wage records are traditionally maintained by a supervisor or the owner in a handwritten notebook, supplemented by verbal agreements about overtime, advances and bonuses.

This arrangement works only while the workforce is small and trust is high. As a business grows, handwritten records become slow to total, difficult to verify, easy to alter and nearly impossible to audit. Meanwhile, mainstream enterprise HR suites are priced, designed and documented for white-collar organisations with desks, corporate email and biometric hardware budgets – none of which describes a brick kiln, a building site or a neighbourhood workshop.

1.2 Motivation

The motivation for WorkLedger is both social and commercial. Socially, workers deserve a clear, tamper-resistant record of the days they worked and the wages they earned. Commercially, owners lose money through calculation mistakes, proxy attendance, forgotten advances and the sheer time spent reconciling notebooks at month-end. Smartphone penetration among workers and owners is now high enough that a lightweight web application can replace the ledger without requiring any specialised hardware.

1.3 Why Digitise Worker Management?

- **Accuracy:** machine arithmetic removes transcription and totalling errors.
- **Accountability:** each entry carries a server timestamp and an actor, making silent edits visible.
- **Speed:** month-end payroll that once took days is generated in seconds.
- **Trust:** workers can see exactly how their pay was derived, reducing disputes.
- **Decision support:** owners gain real-time attendance and cost visibility across the whole workforce.

1.4 Report Organisation

Chapter 1 introduces the problem and scope. Chapter 2 reviews existing approaches and establishes feasibility. Chapters 3 to 5 specify requirements, architecture and data design. Chapter 6 details the RBAC, attendance and payroll modules. Chapters 7 and 8 justify the technology stack and describe quality assurance. Chapter 9 presents the business case and Chapter 10 the future roadmap, followed by the conclusion and references.

Chapter 1 (continued) – Problem Statement

Paper-based labour ledgers suffer from structural weaknesses that cannot be fixed by working harder or being more careful. The weaknesses arise from the medium itself: paper has no validation, no access control, no search, no backup and no history.

Operational Inefficiencies

- Daily roll-calls consume supervisor time and are frequently completed from memory at the end of the day.
- Month-end settlement requires manually counting marks for every worker, often under time pressure on the same day wages are due.
- Locating a past record – for example, to resolve a dispute about a specific date – means searching physical notebooks that may be lost, damaged or unreadable.
- Owners who are off-site have no live view of who is present, and therefore cannot make staffing decisions in time.

Financial Leakage

Financial leakage is the gap between what a business **should** pay and what it **actually** pays. In paper systems it arises from several repeatable mechanisms:

- **Proxy attendance (“buddy punching”):** one worker marks another as present; no evidence exists to challenge the entry.
- **Ghost or duplicated entries:** the same day is recorded twice, or an absent or departed worker remains on the list.
- **Arithmetic and rate errors:** wrong day counts, overtime multipliers misapplied or an outdated daily rate used.
- **Untracked advances:** cash advances are noted informally and forgotten at settlement, or recovered twice.
- **Disputed settlements:** with no shared source of truth, disagreements are resolved by negotiation rather than evidence, often at the owner’s cost or the worker’s.

Human Error and Data Integrity

Handwritten data is exposed to illegible writing, overwriting, torn pages and loss of the ledger itself. There is no mechanism to prevent a date from being skipped, a worker from being counted twice, or an entry from being altered after the fact. Because there is no audit trail, even honest corrections create suspicion.

Table 4: Problems of paper-based ledgers

Problem	Root Cause	Business Impact
Proxy / false attendance	No identity evidence at time of marking	Direct wage loss; unfairness to honest workers
Calculation mistakes	Manual arithmetic, inconsistent rules	Over- or under-payment; worker dissatisfaction

Problem	Root Cause	Business Impact
Lost or damaged records	Single physical copy, no backup	Irrecoverable history; legal exposure
Advance mismanagement	Informal, unlinked records	Double recovery or non-recovery
Slow month-end process	Manual compilation	Administrative cost; delayed wages
No owner visibility	Information only on paper at the site	Late decisions; weak supervision

WorkLedger addresses each row of the table above by moving the source of truth from a notebook to a validated, access-controlled, backed-up relational database.

Chapter 1 (continued) – Project Objectives & Scope

General Objective

To design, implement and evaluate a secure, responsive and extensible web application that digitises daily attendance recording and monthly payroll calculation for labour-intensive small and medium-sized businesses.

Specific Objectives

1. Implement secure authentication and role-based access control separating **Owner** and **Worker** capabilities.
2. Provide a daily attendance module that records one verified entry per worker per day using a server-generated timestamp.
3. Automate payroll calculation, including daily-rate pay, half-days, paid leave, overtime, bonuses, advances and deductions.
4. Model the domain in a normalised PostgreSQL schema managed through Prisma migrations, with referential integrity and uniqueness constraints.
5. Deliver a mobile-first interface using Tailwind CSS that is usable by non-technical users on low-end smartphones.
6. Establish a testing and troubleshooting methodology covering edge cases, connection-pool behaviour and database runtime errors.
7. Document a credible commercial value proposition and a scalable roadmap suitable for SaaS deployment.

Project Scope

Table 5: Functional boundaries of WorkLedger

In Scope (Current Release)	Out of Scope (Planned for Later)
Owner and Worker authentication with RBAC	Native iOS / Android applications
Daily attendance logging with optional selfie evidence	Biometric and facial-recognition hardware integration
Owner dashboard: workforce list, rates, date filters, payroll totals	Multi-tenant billing and subscription management
Monthly payroll generation with adjustments	Direct bank / mobile-financial-service disbursement
Attendance and payroll history for each worker	Automated SMS / push notifications
Responsive web interface	Statutory tax and provident-fund filing

Long-Term Vision

The long-term vision is to evolve WorkLedger from a single-business tool into a multi-tenant SaaS platform: a worker-centric “digital employment record” that offers mobile access, automated notifications, geofenced attendance, multi-site management, analytics and – with regulatory compliance – integrated wage disbursement. Chapter 10 details the staged path toward this vision.

Chapter 2 – Literature Review & Feasibility Study

2.1 Existing Approaches

Workforce time-and-attendance management has been studied extensively in the software-engineering and information-systems literature, and the market offers solutions at several levels of sophistication. For the purpose of this study, existing approaches are grouped into five categories and compared against the needs of small labour-intensive businesses.

Table 6: Comparative analysis of workforce management approaches

Criterion	Paper Ledger	Spreadsheet	Enterprise HRMS	Biometric Hardware	WorkLedger
Setup cost	Very low	Low	High	High (devices)	Low
Ease of use for workers	High	Medium	Medium	High	High
Tamper resistance	None	Low	High	High	Medium–High
Automated payroll	No	Partial (manual formulas)	Yes	Partial	Yes
Remote owner visibility	No	Limited	Yes	Limited	Yes
Role-based access	No	Weak	Yes	Vendor-specific	Yes
Suited to daily-wage labour	Yes	Partly	Rarely	Partly	Yes
Audit trail	None	Weak	Strong	Strong	Strong

2.2 Gap Analysis

Paper and spreadsheets are cheap but unverifiable. Enterprise HRMS products and biometric systems provide control but are over-engineered and over-priced for a contractor with thirty workers on a building site. The gap WorkLedger targets is the **middle ground**: the auditability and automation of an enterprise system with the simplicity, low cost and mobile-first design of a notebook. This positioning is consistent with established guidance that software for an under-served user group should be shaped around its real workflow rather than adapted from enterprise tooling [7].

2.3 Relevant Technical Literature

- **Role-based access control** formalised by Sandhu et al. [6] provides the model used to separate Owner and Worker privileges.
- **Representational State Transfer (REST)** as described by Fielding [5] underpins the request/response design of the API layer.
- **Relational data modelling and transactional integrity**, as discussed by Kleppmann [9], justify PostgreSQL’s constraints and ACID guarantees for financial records.
- **Password storage** using adaptive hashing, introduced by Provos and Mazières [15], motivates the use of bcrypt-style hashing for credentials.

- **Web application security** risks catalogued by OWASP [10] guide input validation, authentication and access-control design.

2.4 Feasibility Study

Table 7: Feasibility assessment

Dimension	Assessment	Verdict
Technical	All components (Next.js, Prisma, PostgreSQL, Tailwind) are mature, well-documented and free of licence cost. A single full-stack TypeScript codebase reduces integration risk.	Feasible
Economic	Hosting, database and tooling can run on free or low-cost tiers during development; production cost scales with usage. Benefits far exceed costs (Chapter 9).	Feasible
Operational	Interface is designed for non-technical users on low-end phones; training requirement is minimal.	Feasible
Schedule	The modular design allows a working vertical slice within one academic term, with extras added incrementally.	Feasible
Legal / Ethical	Selfie images and phone numbers are personal data; consent, minimal retention and access control are required. Wage records should respect applicable labour law such as the Bangladesh Labour Act 2006 [11].	Feasible with safeguards

Chapter 3 – System Analysis & Requirements

Requirements were elicited from the problem analysis in Chapter 1 and structured following the principles of IEEE Std 830 for software requirements specifications [12]. Each requirement carries an identifier for traceability to design and test cases.

3.1 Actors

Table 8: System actors

Actor	Description	Primary Goals
Owner	Business owner or administrator; one or more per business	Manage workers and rates, review attendance, generate and approve payroll
Worker	Employee whose attendance and pay are tracked	Log attendance, view personal history and salary statements
System	Scheduled and server-side processes	Validate, timestamp, calculate and persist records

3.2 Functional Requirements

Table 9: Functional requirements

ID	Requirement	Priority
FR-01	The system shall authenticate users by phone number / worker ID and password and create a secure session.	High
FR-02	The system shall assign each user exactly one role (OWNER or WORKER) and enforce permissions on the server.	High
FR-03	An Owner shall be able to create, edit, deactivate and set the daily wage rate of Worker accounts.	High
FR-04	A Worker shall be able to record attendance once per calendar day; duplicate submissions shall be rejected.	High
FR-05	The system shall store a server-generated timestamp for each attendance record, ignoring client-supplied time.	High
FR-06	The system shall optionally capture and store a selfie image as evidence with each attendance record.	Medium
FR-07	An Owner shall be able to view, filter by date range and correct attendance records, with the correction logged.	High
FR-08	The system shall support the statuses Present, Half-day, Absent and Paid Leave, plus overtime hours.	High
FR-09	The system shall calculate monthly gross and net pay per worker using the defined payroll formula.	High
FR-10	An Owner shall be able to add bonuses, advances and deductions to a payroll record before finalising.	High
FR-11	The system shall display total payroll liability for a selected period on the Owner dashboard.	High
FR-12	A Worker shall be able to view only their own attendance history and payroll statements.	High
FR-13	The system shall prevent modification of finalised payroll records except through an audited reopen action.	Medium

ID	Requirement	Priority
FR-14	The system shall allow export of attendance and payroll summaries (CSV / printable view).	Low

3.3 Non-Functional Requirements

Table 10: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Dashboard pages shall render within two seconds on a typical 4G connection for up to 200 workers.
NFR-02	Security	Passwords shall be stored using an adaptive hash; all traffic shall use HTTPS; authorisation shall be checked on every server request.
NFR-03	Reliability	Data writes shall be transactional; the database shall be backed up daily with tested restore.
NFR-04	Usability	Core tasks (log attendance, view salary) shall be completable in three taps or fewer after login.
NFR-05	Compatibility	The interface shall be responsive from 360 px mobile screens to desktop and work on current Chrome, Edge, Safari and Firefox.
NFR-06	Maintainability	The codebase shall be fully typed (TypeScript), modular and covered by automated tests for payroll logic.
NFR-07	Scalability	The architecture shall support horizontal scaling of the application tier and connection pooling for the database tier.
NFR-08	Privacy	Personal data (phone, selfies) shall be accessible only to authorised roles and retained according to a defined policy.
NFR-09	Auditability	All attendance corrections and payroll state changes shall record actor and timestamp.

3.4 Constraints and Assumptions

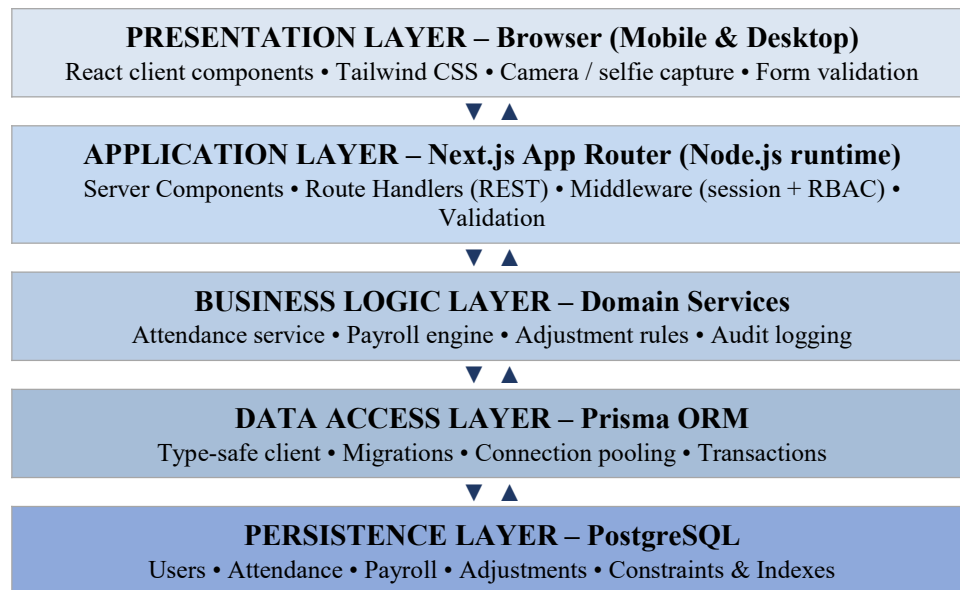
- Users have access to a smartphone or computer with an internet connection during attendance logging.
- A single business (tenant) per deployment in the current release; multi-tenancy is a roadmap item.
- Wages are expressed in a single currency (BDT) and rounded to two decimal places.
- Working-day standard is eight hours; overtime is paid at a configurable multiplier (default 1.5×).

Chapter 4 – System Architecture & Data Flow

4.1 Architectural Overview

WorkLedger follows a layered, client–server architecture implemented as a single full-stack Next.js application. The App Router combines server-rendered React components for data-heavy views, client components for interactive widgets, and route handlers for the JSON API. A shared Prisma client mediates all database access. Figure 1 shows the logical layers.

Figure 1: Layered architecture of WorkLedger



4.2 Client–Server Communication Model

The browser communicates with the server over HTTPS using a request/response model consistent with REST principles [5]. Read-heavy pages (dashboards, history) are rendered on the server so that the browser receives ready-to-display HTML with minimal JavaScript. Mutating actions – logging attendance, adding an adjustment, generating payroll – are sent as authenticated JSON requests to route handlers, which validate input, enforce authorisation, execute the operation inside the database and return a typed response.

4.3 Request Lifecycle

1. The user submits a request (page navigation or API call) over HTTPS with a session cookie.
2. **Middleware** verifies the session token and rejects unauthenticated requests before any application code executes.
3. The **route handler or server component** loads the session, checks that the user’s role is permitted for the resource (RBAC) and, for workers, that the resource belongs to them.
4. Input is **validated and normalised** (types, ranges, date boundaries).

5. The **domain service** executes business rules – for example, rejecting a second attendance entry for the same day or computing payroll.
6. **Prisma** issues parameterised SQL to PostgreSQL, inside a transaction where multiple writes must succeed together.
7. The response is serialised and returned; errors are mapped to meaningful HTTP status codes (400, 401, 403, 404, 409, 500).

4.4 Data Flow Summary

Table 11: Principal data flows

Flow	Source → Destination	Data
Login	Browser → Auth route → users table	Credentials in; session token out
Attendance logging	Worker browser → Attendance route → attendance table	Selfie (optional); server time and status stored
Owner review	attendance table → Server component → Owner browser	Filtered attendance list and summary counts
Payroll generation	attendance + users → Payroll engine → payroll tables	Computed gross, adjustments and net pay
Worker statement	payroll tables → Server component → Worker browser	Own-record salary statement only

4.5 Security Architecture

- **Defence in depth:** middleware, route-level checks and database constraints each independently protect data.
- **Server-side authority:** the server decides time, role and ownership; the client is never trusted for these values.
- **Parameterised queries** via Prisma mitigate SQL injection; output encoding by React mitigates cross-site scripting [10].
- **Secrets management:** database URLs and signing keys are held in environment variables and never committed to source control.

Chapter 5 – Database Design & Schema

5.1 Design Principles

The database is a PostgreSQL instance accessed through Prisma ORM. The design follows third normal form, uses enumerated types for controlled vocabularies, uses exact decimal types for money, and enforces business invariants with database constraints so that integrity does not depend solely on application code.

5.2 Entities and Relationships

Table 12: Entity summary

Entity	Purpose	Key Relationships
User	Owners and workers, credentials, role and daily wage rate	1 User → N Attendance; 1 User → N Payroll; Owner 1 → N Workers (self-relation)
Attendance	One record per worker per calendar day	N Attendance → 1 User; unique (userId, workDate)
Payroll	Monthly salary statement per worker	N Payroll → 1 User; unique (userId, year, month); 1 Payroll → N Adjustment
PayrollAdjustment	Bonus, advance or deduction line items	N Adjustment → 1 Payroll

5.3 Prisma Schema

The following schema is the authoritative definition of the data model.

Listing 1: schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url       = env("DATABASE_URL")    // pooled connection
  directUrl = env("DIRECT_URL")      // direct connection for migrations
}

enum Role { OWNER WORKER }
enum AttendanceStatus { PRESENT HALF_DAY ABSENT PAID_LEAVE }
enum PayrollStatus { DRAFT FINALIZED PAID }
enum AdjustmentType { BONUS ADVANCE DEDUCTION }

model User {
  id          String      @id @default(cuid())
  name        String
  phone       String      @unique
  passwordHash String
  role        Role         @default(WORKER)
  dailyRate   Decimal      @default(0) @db.Decimal(10, 2)
  isActive    Boolean      @default(true)
  ownerId     String?
  owner       User?        @relation("OwnerWorkers", fields: [ownerId], references:
[id])
  workers     User[]        @relation("OwnerWorkers")
  attendances Attendance[]
  payrolls    Payroll[]
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
}
```

```

    @@index([role, isActive])
}

model Attendance {
  id          String          @id @default(cuid())
  userId      String
  user        User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
  workDate    DateTime         @db.Date
  checkInAt   DateTime         @default(now())
  checkOutAt   DateTime?
  status       AttendanceStatus @default(PRESENT)
  overtimeHours Decimal         @default(0) @db.Decimal(4, 2)
  selfieUrl    String?
  verifiedById String?
  note         String?

  @@unique([userId, workDate])
  @@index([workDate])
}

model Payroll {
  id          String          @id @default(cuid())
  userId      String
  user        User            @relation(fields: [userId], references: [id])
  periodYear  Int
  periodMonth Int
  rateSnapshot Decimal         @db.Decimal(10, 2)
  paidDays     Decimal         @db.Decimal(5, 2)
  overtimeHours Decimal         @db.Decimal(6, 2)
  grossPay     Decimal         @db.Decimal(12, 2)
  totalAdditions Decimal       @default(0) @db.Decimal(12, 2)
  totalDeductions Decimal       @default(0) @db.Decimal(12, 2)
  netPay       Decimal         @db.Decimal(12, 2)
  status       PayrollStatus   @default(DRAFT)
  generatedAt   DateTime       @default(now())
  paidAt        DateTime?
  adjustments   PayrollAdjustment[]

  @@unique([userId, periodYear, periodMonth])
}

model PayrollAdjustment {
  id          String          @id @default(cuid())
  payrollId   String
  payroll     Payroll         @relation(fields: [payrollId], references: [id], onDelete:
Cascade)
  type        AdjustmentType
  amount       Decimal         @db.Decimal(12, 2)
  reason       String?
  createdAt    DateTime       @default(now())
}

```

5.4 Key Design Decisions

Table 13: Schema design decisions

Decision	Rationale
Composite unique key (userId, workDate) on Attendance	Makes duplicate attendance impossible at the database level, even under concurrent requests.
Decimal(10,2) / Decimal(12,2) for money	Avoids floating-point rounding errors that are unacceptable in payroll.
rateSnapshot stored on Payroll	Later changes to a worker's rate never alter historical payroll.

Decision	Rationale
Date-only column (@db.Date) for workDate	Removes time-of-day and time-zone ambiguity when grouping by day.
Enums for status and role	Constrains values and gives compile-time type safety through Prisma Client.
Self-relation for Owner → Workers	Prepares the model for multi-business ownership without a separate tenant table.
Cascade delete on Attendance, restrict on Payroll	Attendance is derived data; payroll is a financial record and must not vanish silently.
Indexes on workDate and (role, isActive)	Accelerates the date-range filters and active-worker lists used by the dashboard.

Chapter 6 – Core Features: Role-Based Access Control

WorkLedger implements the role-based access control model of Sandhu et al. [6]: permissions are attached to roles, and users acquire permissions only through the role they hold. Two roles exist – **OWNER** and **WORKER** – producing two distinct dashboards and two distinct sets of server-side permissions.

6.1 Permission Matrix

Table 14: Role–permission matrix

Capability	Owner	Worker
View all workers and their rates	Yes	No
Create / edit / deactivate workers; set daily rate	Yes	No
Log own attendance	Optional	Yes
View own attendance history	Yes	Yes
View other workers' attendance	Yes	No
Correct attendance (audited)	Yes	No
Generate / finalise payroll	Yes	No
View total payroll liability	Yes	No
View own salary statement	Yes	Yes (own only)

6.2 Owner Dashboard

- Summary cards: workers present today, absent today, month-to-date payroll liability.
- Worker directory with daily rate, status and quick actions.
- Date-range filter for attendance review, including selfie evidence and correction controls.
- Payroll console: generate period, add adjustments, finalise and mark as paid.

6.3 Worker Dashboard

- One-tap “Mark attendance” action with optional selfie capture and confirmation of the server-recorded time.
- Personal calendar of recorded days with status and overtime.
- Salary statement showing days, overtime, adjustments and net pay, so every figure is explainable.

6.4 Enforcement in Code

Hiding a button is not security. WorkLedger enforces roles on the server in a reusable guard that every protected route handler and server component calls before touching data.

Listing 2: Role guard and ownership check (TypeScript)

```
import { getSession } from "@lib/session";
import { prisma } from "@lib/prisma";
```

```

type Role = "OWNER" | "WORKER";

export async function requireRole(...allowed: Role[]) {
  const session = await getSession(); // verifies signed cookie
  if (!session) throw new HttpError(401, "Not authenticated");
  if (!allowed.includes(session.role))
    throw new HttpError(403, "Forbidden for role " + session.role);
  return session;
}

// Workers may read only their own payroll; owners may read any.
export async function getPayrollForViewer(payrollId: string) {
  const session = await requireRole("OWNER", "WORKER");
  const payroll = await prisma.payroll.findUnique({ where: { id: payrollId } });
  if (!payroll) throw new HttpError(404, "Not found");
  if (session.role === "WORKER" && payroll.userId !== session.userId)
    throw new HttpError(403, "Forbidden");
  return payroll;
}

```

Middleware additionally redirects unauthenticated visitors to the login page and routes each authenticated user to the dashboard matching their role, so an owner and a worker never see each other's navigation.

Chapter 6 (continued) – Daily Attendance Tracking Module

7.1 Design Goals

The attendance module must be fast enough to use at a site gate with a queue of workers, trustworthy enough to settle a dispute, and strict enough that the same day cannot be recorded twice. Its three guiding principles are **server authority**, **single record per day** and **evidence on demand**.

7.2 Workflow

1. The worker signs in and opens the Worker dashboard, which shows whether today is already marked.
2. The worker taps **Mark Attendance**; the browser optionally captures a selfie using the device camera.
3. The client posts the request; the server authenticates the session and confirms the WORKER role.
4. The server computes today's **work date** in the business time zone (Asia/Dhaka) – the client's clock is ignored.
5. The server attempts to insert a record; the unique constraint on (userId, workDate) guarantees at most one row per day.
6. On success the worker sees a confirmation with the recorded time; on conflict, a friendly "already marked" message.
7. The owner sees the entry immediately in the live dashboard and may review the selfie, adjust status or add overtime.

7.3 Implementation

Listing 3: Attendance route handler

```
import { Prisma } from "@prisma/client";
import { prisma } from "@lib/prisma";
import { requireRole } from "@lib/rbac";
import { businessDate } from "@lib/time"; // returns date-only value in
Asia/Dhaka

export async function POST(req: Request) {
  const session = await requireRole("WORKER");
  const { selfieUrl } = await req.json();

  try {
    const record = await prisma.attendance.create({
      data: {
        userId: session.userId,
        workDate: businessDate(new Date()), // server time, never client time
        status: "PRESENT",
        selfieUrl: selfieUrl ?? null,
      },
    });
    return Response.json(record, { status: 201 });
  } catch (e) {
    if (e instanceof Prisma.PrismaClientKnownRequestError && e.code === "P2002") {
      return Response.json({ error: "Attendance already recorded today" }, { status:
409 });
    }
    throw e;
  }
}
```

```
}  
}
```

7.4 Verification Layers

Table 15: Attendance verification layers

Layer	Mechanism	Threat Mitigated
Identity	Authenticated session bound to a single worker account	Anonymous or forged entries
Time integrity	Server-generated timestamp and date	Back-dating or clock manipulation
Uniqueness	Database unique constraint (userId, workDate)	Duplicate or double-counted days, including race conditions
Visual evidence	Optional selfie stored with the record	Proxy attendance (buddy punching)
Human review	Owner can inspect, correct and annotate entries	Edge cases that automation cannot judge
Audit trail	verifiedById, note and timestamps on corrections	Silent edits; disputes about who changed what

7.5 Edge Cases

- **Midnight boundary:** the work date is derived in the business time zone, so a record made at 00:05 belongs to the new day.
- **Forgotten attendance:** the owner can create a retroactive record flagged with a note and actor identity.
- **Double tap / slow network:** the unique constraint makes the second request harmless; the client treats a 409 as success-with-notice.
- **Camera denied or unavailable:** attendance still records; the entry is simply flagged as “no selfie” for owner review.
- **Half-days and leave:** the owner sets status and overtime, which feed directly into payroll.

Chapter 6 (continued) – Payroll & Salary Calculation Module

8.1 Payroll Formula

Payroll is a deterministic function of the attendance records for the period, the worker's daily rate and any adjustments. For a worker with daily rate *R*, the following definitions apply:

Table 16: Payroll calculation formulas

Component	Formula	Notes
Paid days (D)	$D = \text{Present} + \text{PaidLeave} + 0.5 \times \text{HalfDays}$	Absent days contribute zero
Base pay	$\text{Base} = D \times R$	Uses the rate snapshot taken at generation time
Hourly rate	$H = R \div 8$	Standard eight-hour working day
Overtime pay	$\text{OT} = \text{OvertimeHours} \times H \times m$	m = overtime multiplier, default 1.5
Gross pay	$\text{Gross} = \text{Base} + \text{OT}$	
Additions	$A = \Sigma \text{Bonus}$	Entered by owner with reason
Deductions	$X = \Sigma \text{Advance} + \Sigma \text{Deduction}$	Advances recovered at settlement
Net pay	$\text{Net} = \max(0, \text{Gross} + A - X)$	Shortfall may be carried forward as a new advance

8.2 Worked Example

Consider a worker with a daily rate of BDT 700 in a month with 24 present days, 2 half-days, 2 absences, 10 overtime hours, a bonus of BDT 500 and an advance of BDT 2,000.

Table 17: Worked payroll example

Step	Calculation	Result (BDT)
Paid days	$24 + 0.5 \times 2$	25.0 days
Base pay	$24 \times 700 + 2 \times 350$	17,500.00
Overtime pay	$10 \times (700 \div 8) \times 1.5 = 10 \times 131.25$	1,312.50
Gross pay	$17,500.00 + 1,312.50$	18,812.50
Additions	Bonus	+ 500.00
Deductions	Advance	- 2,000.00
Net pay	$18,812.50 + 500.00 - 2,000.00$	17,312.50

8.3 Calculation Engine

The calculation is implemented as a pure function with no database access, so it can be unit-tested exhaustively and reused unchanged by the future mobile app and reporting features.

Listing 4: Pure payroll function

```
const HOURS_PER_DAY = 8;
const OT_MULTIPLIER = 1.5;
const round2 = (n: number) => Math.round((n + Number.EPSILON) * 100) / 100;

export interface PayrollInput {
  dailyRate: number;
```

```

present: number; halfDays: number; paidLeave: number;
overtimeHours: number;
adjustments: { type: "BONUS" | "ADVANCE" | "DEDUCTION"; amount: number }[];
}

export function computePayroll(i: PayrollInput) {
  const paidDays = i.present + i.paidLeave + i.halfDays * 0.5;
  const base     = paidDays * i.dailyRate;
  const overtime = i.overtimeHours * (i.dailyRate / HOURS_PER_DAY) * OT_MULTIPLIER;
  const gross    = base + overtime;

  const additions = sum(i.adjustments.filter(a => a.type === "BONUS"));
  const deductions = sum(i.adjustments.filter(a => a.type !== "BONUS"));
  const net       = Math.max(0, gross + additions - deductions);

  return {
    paidDays, grossPay: round2(gross),
    totalAdditions: round2(additions), totalDeductions: round2(deductions),
    netPay: round2(net),
  };
}

const sum = (xs: { amount: number }[]) => xs.reduce((s, x) => s + x.amount, 0);

```

8.4 Monthly Processing Lifecycle

Table 18: Payroll status lifecycle

State	Description	Allowed Actions
DRAFT	Generated from attendance; recalculated whenever attendance or adjustments change	Add / remove adjustments, regenerate, finalise
FINALIZED	Locked; figures frozen and visible to the worker	Mark as paid; audited reopen only
PAID	Settlement completed; paidAt timestamp recorded	View, export; no edits

Generation runs inside a single database transaction: attendance for the period is aggregated, the payroll row is upserted on the unique key (userId, year, month), and adjustments are re-linked. If any step fails, nothing is written, so a partially generated payroll can never exist.

Chapter 7 – Technology Stack & Performance Optimisation

9.1 Stack Overview

Table 19: Technology stack and rationale

Layer	Technology	Reason for Selection
Framework	Next.js (App Router, Turbopack)	Full-stack React with server rendering, route handlers and middleware in one codebase; fast development builds
UI library	React with TypeScript	Component model, large ecosystem, compile-time type checking
Styling	Tailwind CSS	Utility-first, consistent design tokens, small production CSS, rapid responsive design
ORM	Prisma	Schema-first modelling, generated type-safe client, versioned migrations
Database	PostgreSQL	ACID transactions, rich constraints, exact decimal arithmetic, mature tooling
Authentication	Signed session cookies, adaptive password hashing	Server-verifiable sessions; resistant to offline password cracking [15]

9.2 Why Next.js with the App Router and Turbopack

- **Server Components** render dashboards on the server, sending HTML rather than large JavaScript bundles – important for low-end phones and weak networks.
- **Route handlers and middleware** place the API, authentication and RBAC in the same project as the UI, removing a separate backend deployment during early stages.
- **Streaming and nested layouts** let the shell of a dashboard appear immediately while heavier data sections load.
- **Turbopack** provides an incremental, Rust-based bundler that gives fast cold starts and near-instant hot updates in development, shortening the edit–test loop on student-grade hardware.

9.3 Why Tailwind CSS

Tailwind's utility classes keep styling next to markup, eliminating context switching and dead CSS. Only the classes actually used are emitted, so production stylesheets stay small. Responsive variants (for example `'sm:'`, `'md:'`) make the mobile-first requirement (NFR-05) straightforward to satisfy, and design tokens for colour, spacing and typography keep the owner and worker interfaces visually consistent.

9.4 Why Prisma

Prisma turns the schema in Listing 1 into a fully typed client. A misspelled field or an incorrect relation is a compile-time error rather than a production failure, which matters most in financial code. Migrations are versioned alongside the source so that every environment reaches the same schema deterministically, and transactions (`prisma.$transaction`) provide the atomicity that payroll generation requires.

9.5 Performance Optimisation Strategies

Table 20: Performance optimisation measures

Area	Technique	Benefit
Rendering	Server Components for read-heavy pages; client components only for interactive elements	Smaller JavaScript payload, faster first paint
Database	Indexes on workDate and (role, isActive); selective field queries with `select`	Faster filtered lists; less data transferred
Aggregation	Group-by and aggregate queries in SQL rather than loading rows into memory	Constant-memory payroll and dashboard totals
Connections	Singleton Prisma client; pooled connection string in production	No connection exhaustion under load or hot reload
Caching	Cached, revalidated dashboard summaries; static assets served via CDN	Lower database load, faster repeat views
Assets	Compressed, size-limited selfie images; lazy loading in lists	Lower bandwidth on mobile networks
Type safety	End-to-end TypeScript from schema to UI	Fewer runtime defects; safer refactoring

9.6 Reliability and Type Safety

Reliability comes from combining static and runtime guarantees. TypeScript and Prisma's generated types catch structural errors at build time; input validation catches malformed requests at run time; and PostgreSQL constraints catch anything that slips through. Each layer assumes the others can fail, which is the essence of defence in depth.

Chapter 8 – Testing, Quality Assurance & Troubleshooting

10.1 Test Strategy

Table 21: Testing levels

Level	Scope	Examples
Unit	Pure functions	computePayroll with zero days, only half-days, overtime only, adjustments exceeding gross, rounding
Integration	Route handlers with a test database	Duplicate attendance returns 409; worker cannot read another worker's payroll (403)
End-to-end	Browser flows	Worker logs in and marks attendance; owner sees the entry and generates payroll
Security	Authorisation and input	Role escalation attempts, tampered cookies, malformed JSON, injection strings
Usability	Real users on real phones	Task completion time and error rate for non-technical testers
Regression	Automated suite on every change	Previously fixed defects stay fixed

10.2 Edge Cases Handled

Table 22: Edge cases and handling

Edge Case	Handling
Two attendance requests at the same instant	Unique constraint guarantees one row; second request receives 409
Attendance near midnight or across time zones	Work date derived on the server in Asia/Dhaka
Worker's rate changes mid-month	Payroll uses the snapshot at generation; owner may regenerate drafts intentionally
Advances exceed earnings	Net floored at zero; shortfall recorded as a carried-forward advance
Worker deactivated mid-month	Historical attendance and payroll retained; no new records permitted
Attendance edited after payroll finalised	Blocked unless payroll is reopened through an audited action
Months with different day counts or leap years	Payroll is based on recorded paid days, not a fixed divisor
Network failure during submission	Client shows retry; idempotent server behaviour prevents duplicates

10.3 Connection Pool Management

A frequent source of failure in Next.js with Prisma is **connection exhaustion**. In development, hot reloading re-evaluates modules; if each evaluation creates a new `PrismaClient`, each client opens its own pool and PostgreSQL eventually refuses connections with an error such as “too many clients already.” In production, serverless or

multi-instance deployments multiply the problem. WorkLedger addresses this with a singleton client:

Listing 5: Singleton Prisma client (lib/prisma.ts)

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: process.env.NODE_ENV === "development" ? ["warn", "error"] : ["error"],
  });

// Reuse the same instance across hot reloads in development
if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

- Use a **pooled connection string** (for example through PgBouncer or a provider's pooler) as DATABASE_URL and a **direct** string as DIRECT_URL for migrations.
- Limit pool size per instance (`connection\_limit` parameter) so that $\text{instances} \times \text{pool size}$ stays below the database's maximum connections.
- Avoid creating clients inside request handlers or components; always import the shared instance.
- Disconnect explicitly only in scripts and tests, not in the web server process.

10.4 Debugging Database Runtime Errors

Table 23: Common database runtime errors and fixes

Symptom / Error	Likely Cause	Resolution
P1001 – Can't reach database server	Wrong host or port, database paused or sleeping, firewall or network issue	Verify DATABASE_URL, wake or restart the database, check network and SSL settings
P1017 – Server has closed the connection	Idle connection dropped by server or pooler	Use pooled URL, enable connection retry, reduce idle time
“Too many clients already”	Multiple PrismaClient instances or oversized pools	Apply singleton pattern; set connection_limit; use a pooler
P2002 – Unique constraint failed	Duplicate attendance, phone number or payroll period	Catch and map to HTTP 409 with a clear message
P2025 – Record not found	Update or delete of a missing row	Check existence first or handle and return 404
P2003 – Foreign key constraint failed	Referencing a user or payroll that does not exist	Validate IDs; check deletion order
Prisma Client not initialised / out of date	`prisma generate` not run after a schema change	Run `prisma generate` and restart the dev server
Migration drift or failure	Schema changed manually or pooled URL used for migrations	Use DIRECT_URL for migrations; reconcile with `prisma migrate`
Slow queries	Missing index or unbounded result sets	Add indexes, paginate, use `select` and database-side aggregation

A recommended debugging routine is: (1) read the exact Prisma error code; (2) reproduce with the same environment variables; (3) enable query logging; (4) check connection counts in the database; (5) add a regression test once the cause is found.

Chapter 9 – Business Impact, ROI & Buyer Value

11.1 Value to Business Owners

Table 24: Business benefits

Benefit	Operational Effect	Financial Effect
Reduced proxy attendance	Selfie evidence and server timestamps deter false entries	Lower wage leakage
Error-free calculation	Automated payroll replaces manual totals	Fewer over/under-payments and disputes
Time saved on administration	Month-end settlement in minutes	Lower clerical cost; faster payment
Real-time visibility	Owner sees attendance from anywhere	Better staffing and cost control
Advance tracking	Advances recorded and recovered automatically	No double or missed recovery
Auditable records	Searchable history for any worker and date	Reduced legal and reputational risk
Worker trust	Workers can see how pay was derived	Lower turnover and conflict

11.2 Illustrative ROI Model

The model below uses **illustrative assumptions** for a business with 40 workers. Actual figures vary by business and should be validated during a pilot; the structure of the calculation is the useful part.

Table 25: Illustrative return on investment (40-worker business)

Item	Assumption / Calculation	Value (BDT)
Monthly payroll	40 workers × BDT 18,000 average	720,000
Leakage before WorkLedger	4% of payroll (assumed)	28,800 / month
Leakage after WorkLedger	1% of payroll (assumed)	7,200 / month
Leakage savings	28,800 – 7,200	21,600 / month
Admin time savings	24 hours saved × BDT 150 per hour	3,600 / month
Total monthly benefit	21,600 + 3,600	25,200
Subscription / hosting cost	Assumed	5,000 / month
One-time setup and training	Assumed	30,000
Annual benefit	25,200 × 12	302,400
Annual cost (year 1)	5,000 × 12 + 30,000	90,000
Year-1 ROI	$(302,400 - 90,000) \div 90,000$	≈ 236%
Payback period	$30,000 \div (25,200 - 5,000)$	≈ 1.5 months

11.3 Commercial Models

Table 26: Commercial deployment options

Model	Description	Best For
-------	-------------	----------

Model	Description	Best For
SaaS subscription	Monthly fee per business or per active worker; hosted by the vendor	Small and medium businesses wanting zero IT effort
One-time licence + support	Self-hosted deployment with annual support	Larger firms with IT capacity or data-residency needs
White-label / reseller	Branded deployments for contractors' associations and consultancies	Channel partners and industry bodies

11.4 Why Buy or Invest

- **Large, under-served market:** labour-intensive small businesses lack tools designed for them.
- **Low cost to adopt:** no hardware, minimal training, works on existing smartphones.
- **Recurring revenue potential:** per-business or per-worker subscription with strong retention once records live in the system.
- **Defensible data asset:** accumulated attendance and wage history creates switching cost and enables future services such as verified employment records.
- **Modern, maintainable stack:** typed full-stack TypeScript with a clear path to cloud scale reduces technical risk for buyers.

11.5 Risks and Mitigation

Table 27: Adoption risks and mitigation

Risk	Mitigation
Low digital literacy of some workers	Minimal-tap interface; owner-assisted attendance mode; local-language support on the roadmap
Intermittent connectivity at sites	Offline queueing in the planned mobile app; idempotent submissions
Privacy concerns about selfies	Consent notice, access limited to owners, retention limits, optional feature
Owner resistance to change	Pilot with a small group; spreadsheet import; side-by-side month-end comparison

Chapter 10 – Future Roadmap & Scalability

12.1 Phased Roadmap

Table 28: Product roadmap

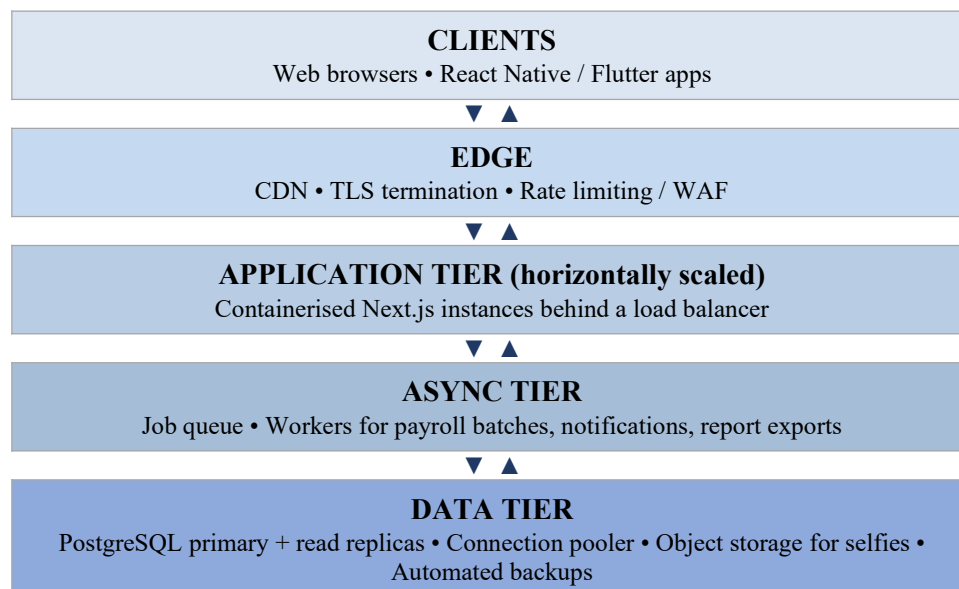
Phase	Focus	Key Deliverables
Phase 1 (Current)	Web MVP	RBAC, attendance, payroll, owner and worker dashboards on PostgreSQL / Prisma
Phase 2	Mobile applications	React Native or Flutter app for workers and owners; offline-first attendance queue; biometric device unlock
Phase 3	Notifications & engagement	Automated SMS and push notifications; attendance reminders; salary-ready alerts; owner exception alerts
Phase 4	Cloud migration & multi-tenancy	Managed PostgreSQL, container deployment, tenant isolation, subscription billing
Phase 5	Intelligence & integration	Geofenced attendance, analytics, anomaly detection, mobile-money / bank disbursement, accounting export

12.2 Mobile Application Integration

The existing route handlers form a ready-made REST API. A React Native or Flutter client can reuse the same endpoints, the same session model (exchanged for token-based authentication) and the same pure payroll function logic on the server. React Native offers code and skill reuse with the existing React/TypeScript team; Flutter offers a single high-performance rendering engine. Either choice enables native camera control, background sync, secure device storage and push-notification delivery.

12.3 Cloud Migration and Scalability Architecture

Figure 2: Target scalable cloud architecture



- **Stateless application instances** allow scaling out simply by adding containers.

- **Connection pooling** keeps total database connections bounded regardless of instance count.
- **Read replicas** serve reporting and dashboard queries, protecting write performance.
- **Object storage** moves selfie images out of the database and application servers, with signed URLs and lifecycle retention rules.
- **Tenant isolation** through a tenant identifier on every row, enforced by query scoping and, optionally, PostgreSQL row-level security.
- **Observability** with structured logs, metrics and alerting to detect slow queries and error spikes early.

12.4 Automated SMS and Push Notifications

Table 29: Notification matrix

Trigger	Recipient	Channel	Message Purpose
Shift start, no attendance recorded	Worker	Push / SMS	Reminder to mark attendance
Attendance recorded or corrected	Worker	Push	Confirmation and transparency
Payroll finalised	Worker	SMS / Push	Salary statement is ready; net amount
Unusual pattern (e.g., many absences)	Owner	Push	Exception alert
Payroll draft awaiting approval	Owner	Email / Push	Approval reminder

Notifications will be dispatched by the asynchronous tier through a provider abstraction so that SMS gateways or push services can be swapped without touching business logic.

Conclusion

13.1 Summary of the Project's Success

WorkLedger demonstrates that a rigorous, auditable labour attendance and payroll system can be delivered as a lightweight web application using modern open-source technology. The project met its objectives: it separates Owner and Worker responsibilities through server-enforced RBAC; it records one verified attendance entry per worker per day under server authority; it calculates payroll through a transparent, testable formula; and it persists everything in a normalised PostgreSQL schema with integrity guaranteed by constraints. The architecture is modular enough to evolve into a mobile-enabled, cloud-scale, multi-tenant SaaS product, and the business analysis shows a compelling value proposition for owners and a credible opportunity for investors.

13.2 Lessons Learned

- **Put invariants in the database.** Application checks can race; a unique constraint cannot.
- **Keep business logic pure.** The payroll function was easiest to test, trust and reuse because it touches no I/O.
- **Never trust the client for time, role or identity.** Server authority removed whole classes of fraud.
- **Connection management is a first-class concern.** Singleton clients and pooled connections prevent a class of confusing runtime failures.
- **Design for non-technical users first.** Reducing taps and words mattered more than adding features.
- **Snapshot financial inputs.** Storing the rate used for each payroll protects history from later changes.

13.3 Limitations

The current release is limited to a single business per deployment, has no offline mode, relies on optional selfies rather than strong biometric verification, and does not yet integrate wage disbursement or notifications. These limitations are addressed in the phased roadmap of Chapter 10.

13.4 Final Remarks

By turning a notebook into a trustworthy system of record, WorkLedger gives owners control and gives workers transparency. The same principle – accurate, shared, verifiable records – is what makes the product valuable to a student evaluating engineering quality and to a buyer evaluating business return.